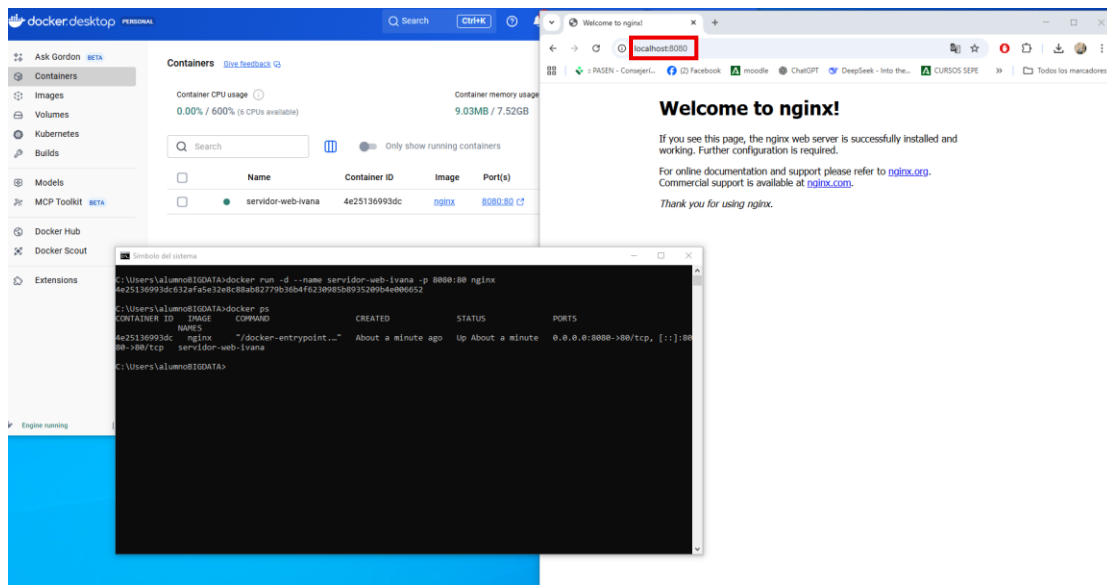


- 1- Crear un contenedor de Docker que tenga la capacidad de ejecutar un servidor web, utilizando una de las imágenes existentes en Docker Hub. Para ello, se debe acceder a Docker Hub, (repositorio en línea de imágenes para contenedores Docker), y buscar una imagen de servidor web que se adecue a las necesidades del proyecto. Luego, se debe utilizar el comando `docker run` junto con el nombre de la imagen seleccionada para crear y ejecutar el contenedor. Por ejemplo, se podría utilizar la imagen "nginx" para crear el contenedor de servidor web.

`docker run -d --name servidor-web-ivana -p 8080:80 nginx`

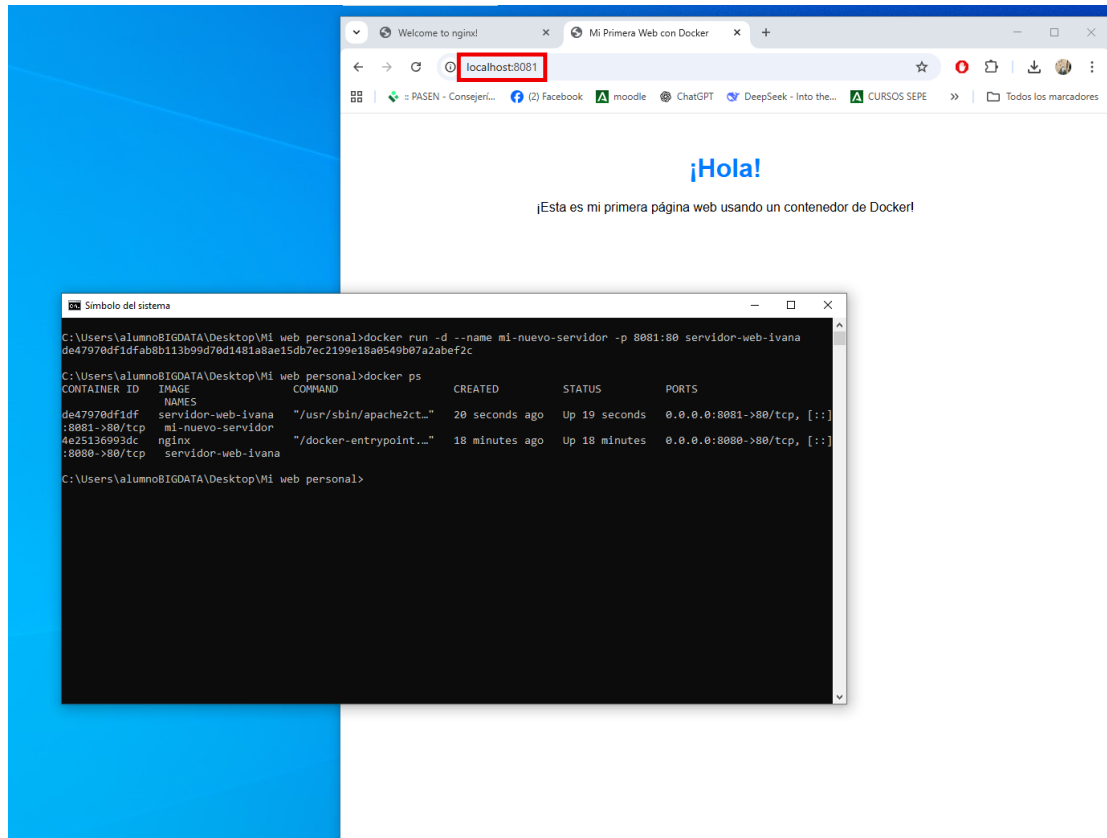


- 2- Crear una imagen de docker mediante un fichero Dockerfile que tenga las siguientes características:
 - 3- Usar una plantilla de Debian.
 - 4- Instalar Apache2. (`apt update && apt install apache2 -y`)
 - 5- Establecer el directorio por defecto en `/var/www/html`
 - 6- Mover una página web que os descarguéis de Internet a dicho directorio.
 - 7- El Puerto 80 debe de estar abierto.
 - 8- Añadir al final del fichero la siguiente instrucción para que apache2 funcione en segundo plano:
CMD `["/usr/sbin/apache2ctl","-DFOREGROUND"]`

- 3- Crear un contenedor a partir del fichero anterior y arrancarlo. Comprobar que la web que has descargado se visualiza en localhost. ¿Qué harías para que la página web pudiera ser visible en localhost:8081?

Simplemente a la hora de crear el contenedor le especifico el puerto 8081 en localhost:

docker run -d --name mi-nuevo-servidor -p 8081:80 servidor-web-ivana



4. ¿Le has dado algún nombre a la imagen? ¿Qué nombre tiene la imagen que has creado? ¿Qué nombre tiene el contenedor?

A la imagen le he dado el nombre de “servidor-web-ivana” y al contenedor le he dado el de “mi-nuevo-servidor”

5. Crear dos nuevos contenedores a partir de la imagen anterior con las siguientes características:
- Un volumen que será compartido entre los dos contenedores y que conectará dentro del contenedor con la ruta `/var/www/html`
 - Los puertos de acceso a los contenedores deben ser 8080 y 8081

docker volumen create volumen-compartido

`docker run -d --name contenedor1 -p 8080:80 -v volumen-compartido:/var/www/html servidor-web-ivana`

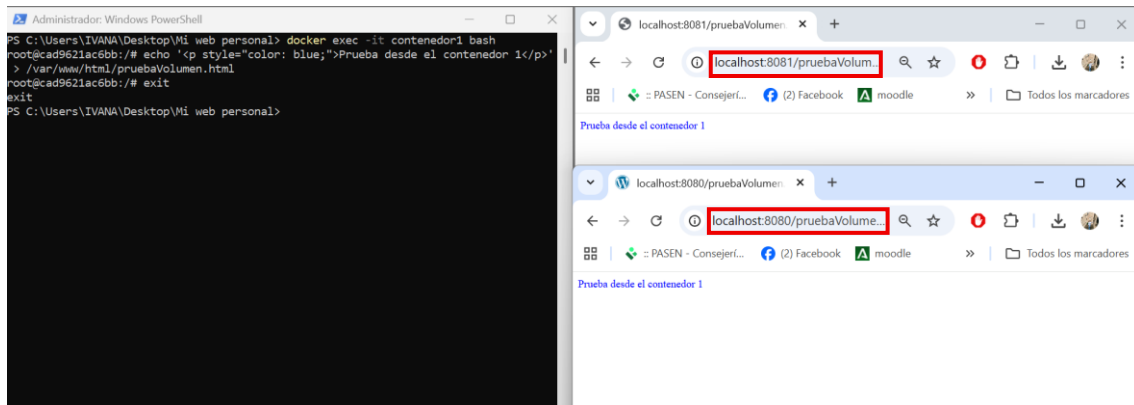
`docker run -d --name contenedor2 -p 8080:80 -v volumen-compartido:/var/www/html servidor-web-ivana`

```
PS C:\Users\IVANA\Desktop\Mi web personal> docker volume create volumen-compartido
volumen-compartido
PS C:\Users\IVANA\Desktop\Mi web personal> docker run -d --name contenedor1 -p 8080:80 -v volumen-compartido:/var/www/html servidor-web-ivana
cad9621ac6bbe46e975d6e6db1ad2e9705d4fb86bd2668bb9d1004e23c306
PS C:\Users\IVANA\Desktop\Mi web personal> docker run -d --name contenedor2 -p 8081:80 -v volumen-compartido:/var/www/html servidor-web-ivana
a65eefc358a537265c6ef8831fdab5d23ecf82a297e302b9fae692cb1e91600a
PS C:\Users\IVANA\Desktop\Mi web personal>
```

Conéctate al primero de los contenedores y crea un fichero pruebaVolumen.html escribe dentro lo siguiente: `<p>Prueba desde el contenedor 1</p>`.

Accede a la ruta <http://localhost:8080/pruebaVolumen.html>.

`docker exec -it contenedor1 bash`
`echo '<p style="color:blue;">Prueba desde el contenedor 1</p>' > /var/www/html/pruebaVolumen.html`

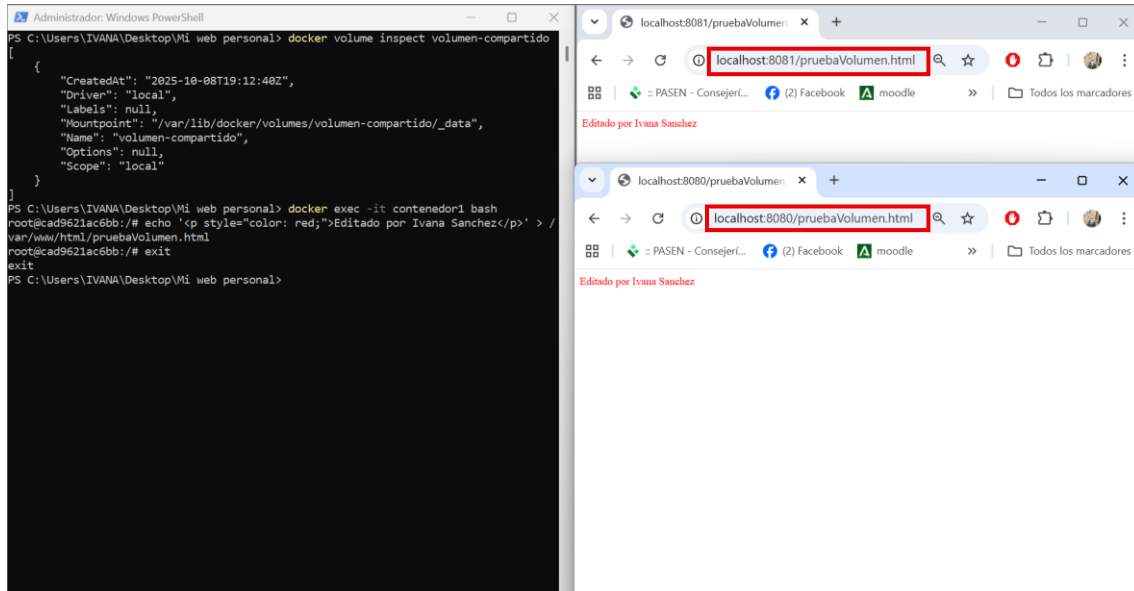


¿Que aparece en el otro contenedor? ¿Existe en la siguiente ruta <http://localhost:8081/pruebaVolumen.html>?

Ambas rutas (contenedor1 y contenedor2) están leyendo los datos del mismo volumen: *volumen-compartido*, el mensaje que he creado en el contenedor1 → *Prueba desde el contenedor 1*.

Edita el fichero desde el host y pon tu nombre, observa los cambios en ambos contenedores

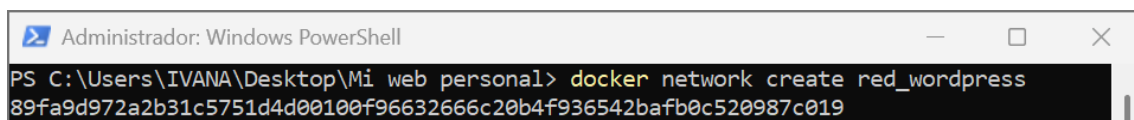
```
docker exec -it contenedor1 bash  
echo '<p style="color:red;">Editado por Ivana Sanchez</p>' >  
/var/www/html/pruebaVolumen.html
```



Desde la PowerShell no he podido hacerlo directamente con un comando, de ahí que simplemente lo haya modificado. Por comando habría que hacerlo por Ubuntu o Linux.

6. Vamos a conectar dos contenedores en red, para ello, crea una red de tipo bridge y a continuación crea los siguientes dos contenedores utilizando la red creada anteriormente.

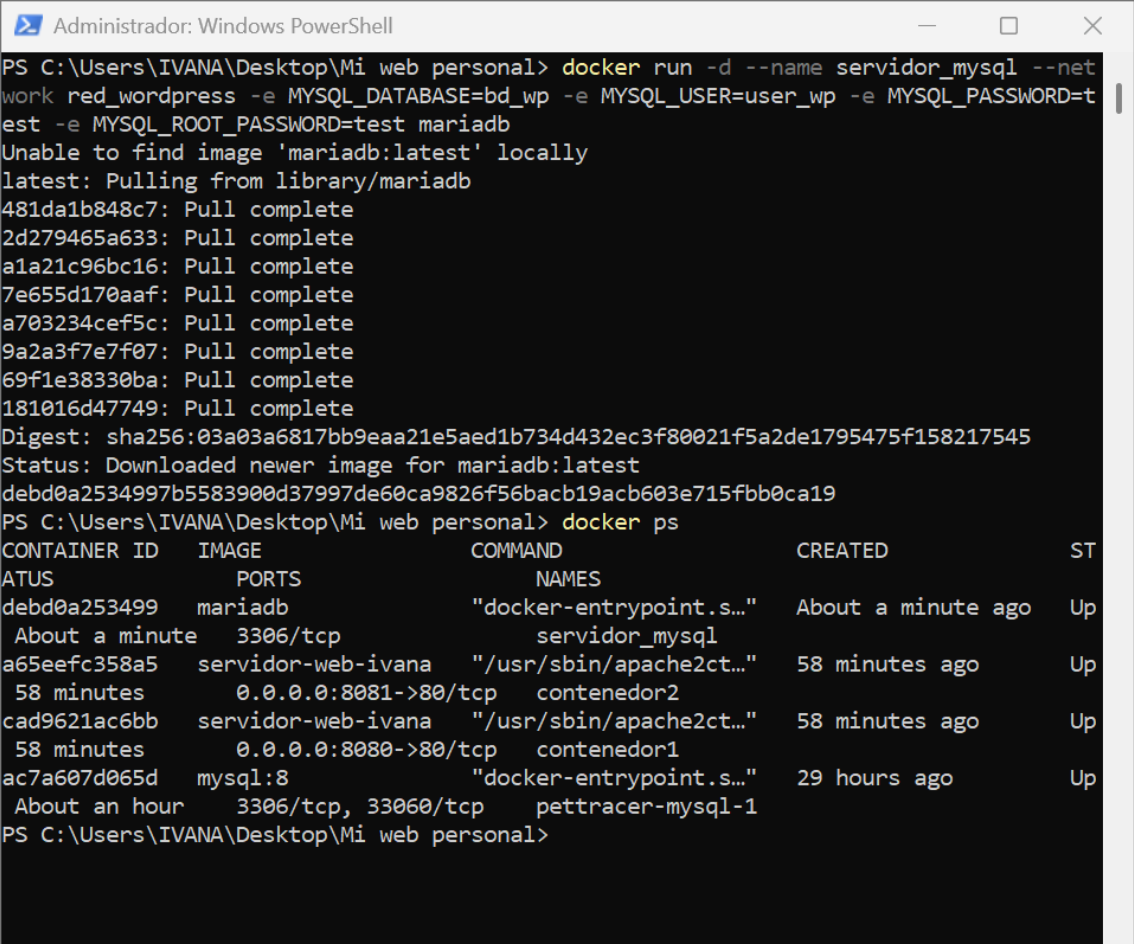
```
docker network create red_wordpress
```



El primer contenedor será una base de datos mariadb y lo llamaremos servidor_mysql:

```
docker run -d --name servidor_mysql --network XXXX -e  
MYSQL_DATABASE=bd_wp -e MYSQL_USER=user_wp -e  
MYSQL_PASSWORD=test -e MYSQL_ROOT_PASSWORD=test mariadb
```

```
docker run -d --name servidor_mysql --network red_wordpress -e  
MYSQL_DATABASE=db_wp -e MYSQL_USER=user_wp -e  
MYSQL_PASSWORD=test -e MYSQL_ROOT_PASSWORD=test mariadb
```

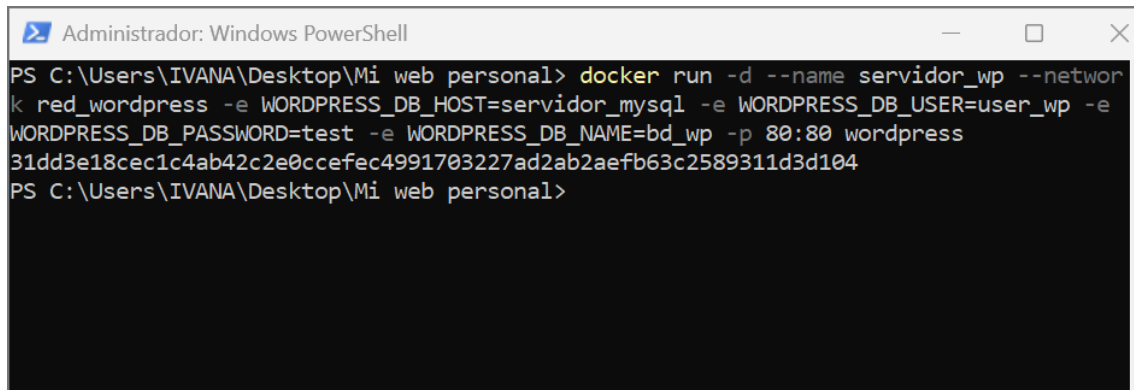


```
Administrador: Windows PowerShell
PS C:\Users\IVANA\Desktop\Mi web personal> docker run -d --name servidor_mysql --net
work red_wordpress -e MYSQL_DATABASE=bd_wp -e MYSQL_USER=user_wp -e MYSQL_PASSWORD=t
est -e MYSQL_ROOT_PASSWORD=test mariadb
Unable to find image 'mariadb:latest' locally
latest: Pulling from library/mariadb
481da1b848c7: Pull complete
2d279465a633: Pull complete
a1a21c96bc16: Pull complete
7e655d170aaf: Pull complete
a703234cef5c: Pull complete
9a2a3f7e7f07: Pull complete
69f1e38330ba: Pull complete
181016d47749: Pull complete
Digest: sha256:03a03a6817bb9eaa21e5aed1b734d432ec3f80021f5a2de1795475f158217545
Status: Downloaded newer image for mariadb:latest
debd0a2534997b5583900d37997de60ca9826f56bacb19acb603e715fbb0ca19
PS C:\Users\IVANA\Desktop\Mi web personal> docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED        STATUS        PORTS                    NAMES
debd0a253499   mariadb              "docker-entrypoint.s..." About a minute Up            3306/tcp               servidor_mysql
a65eefc358a5   servidor-web-ivana   "/usr/sbin/apache2ct..." 58 minutes ago Up            0.0.0.0:8081->80/tcp    contenedor2
cad9621ac6bb   servidor-web-ivana   "/usr/sbin/apache2ct..." 58 minutes ago Up            0.0.0.0:8080->80/tcp    contenedor1
ac7a607d065d   mysql:8              "docker-entrypoint.s..." 29 hours ago   Up            3306/tcp, 33060/tcp      pettracer-mysql-1
PS C:\Users\IVANA\Desktop\Mi web personal>
```

A continuación vamos a crear un contenedor con el nombre servidor_wp, a partir de la imagen wordpress, conectada a la misma red y con las variables de entorno necesarias:

```
docker run -d --name servidor_wp --network XXXX -e
WORDPRESS_DB_HOST=servidor_mysql -e
WORDPRESS_DB_USER=user_wp
-e WORDPRESS_DB_PASSWORD=test -e WORDPRESS_DB_NAME=bd_wp -p
80:80 wordpress
```

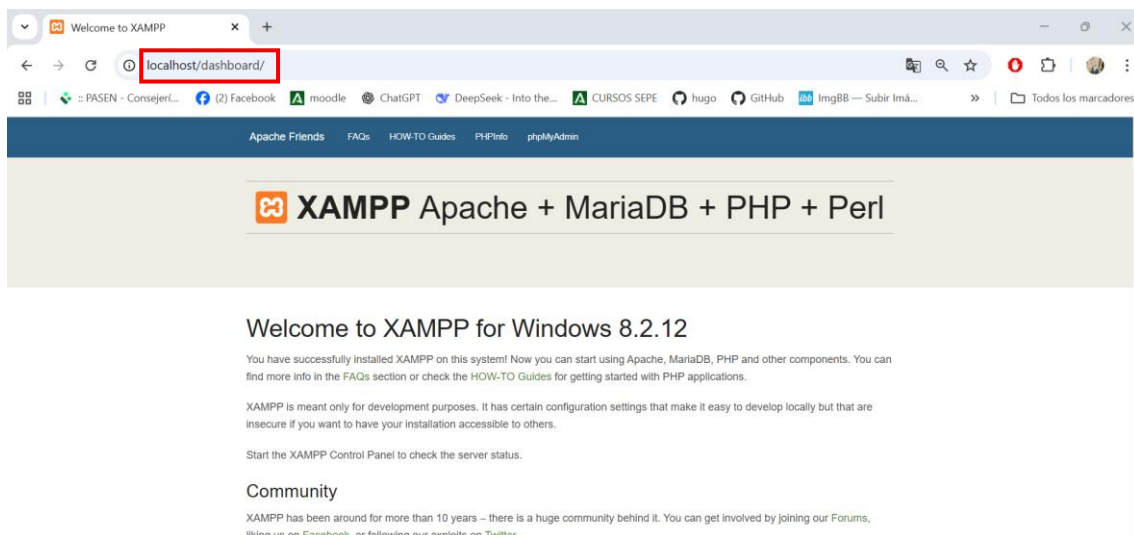
```
docker run -d --name servidor_wp --network red_wordpress -e  
WORDPRESS_DB_HOST=servidor_mysql -e  
WORDPRESS_DB_USER=user_wp -e WORDPRESS_DB_PASSWORD=test -  
e WORDPRESS_DB_NAME=bd_wp -p 80:80 wordpress
```



```
Administrador: Windows PowerShell
PS C:\Users\IVANA\Desktop\Mi web personal> docker run -d --name servidor_wp --network
k red_wordpress -e WORDPRESS_DB_HOST=servidor_mysql -e WORDPRESS_DB_USER=user_wp -e
WORDPRESS_DB_PASSWORD=test -e WORDPRESS_DB_NAME=bd_wp -p 80:80 wordpress
31dd3e18cec1c4ab42c2e0ccfec4991703227ad2ab2aefb63c2589311d3d104
PS C:\Users\IVANA\Desktop\Mi web personal>
```

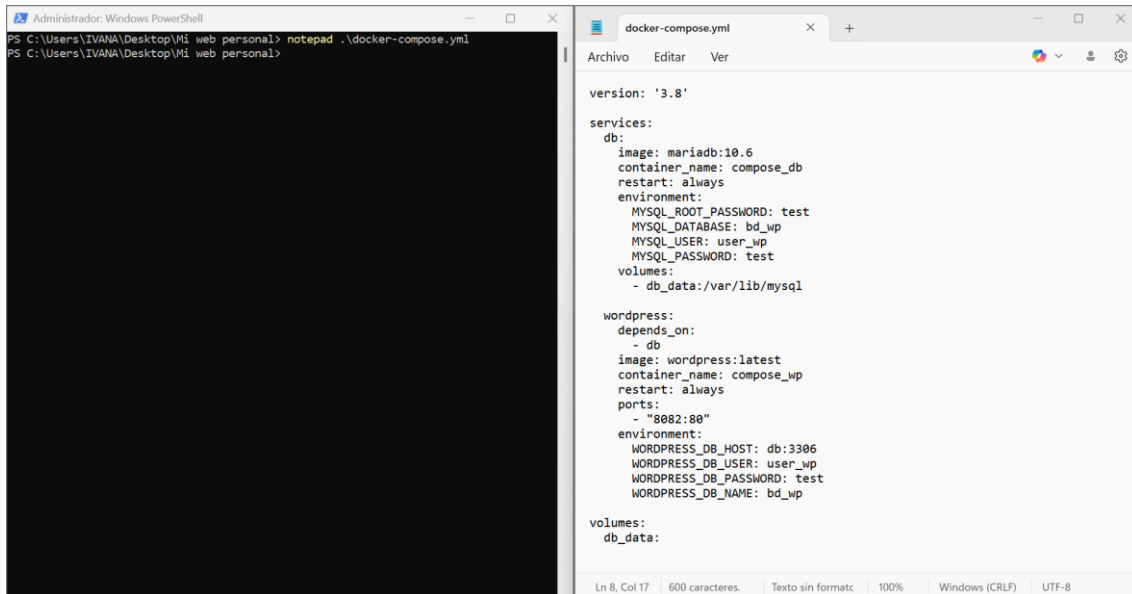
Accede a la ip del servidor docker y comprueba la instalación de wordpress. La variable WORDPRESS_DB_HOST hace referencia a la IP donde está la base de datos mysql ¿Por qué crees que hemos podido poner “servidor_mysql” en la variable WORDPRESS_DB_HOST?

Se puede usar el nombre del contenedor (servidor_mysql) en vez de su IP porque, al conectar ambos contenedores (servidor_mysql y servidor_wp) a la misma red de tipo puente (red_wordpress), Docker configura automáticamente un servicio interno de DNS. Esto permite que los contenedores dentro de esa red se comuniquen usando el nombre que le he dado.



7. Buscar en docker hub la imagen de wordpress y cómo se instala con docker-compose. Generar el fichero correspondiente para arrancarlo en vuestras máquinas.

notepad docker-compose.yml



The screenshot shows two windows. On the left, a Windows PowerShell window with the command `notepad .\docker-compose.yml` executed. On the right, a Notepad window displaying the contents of `docker-compose.yml`.

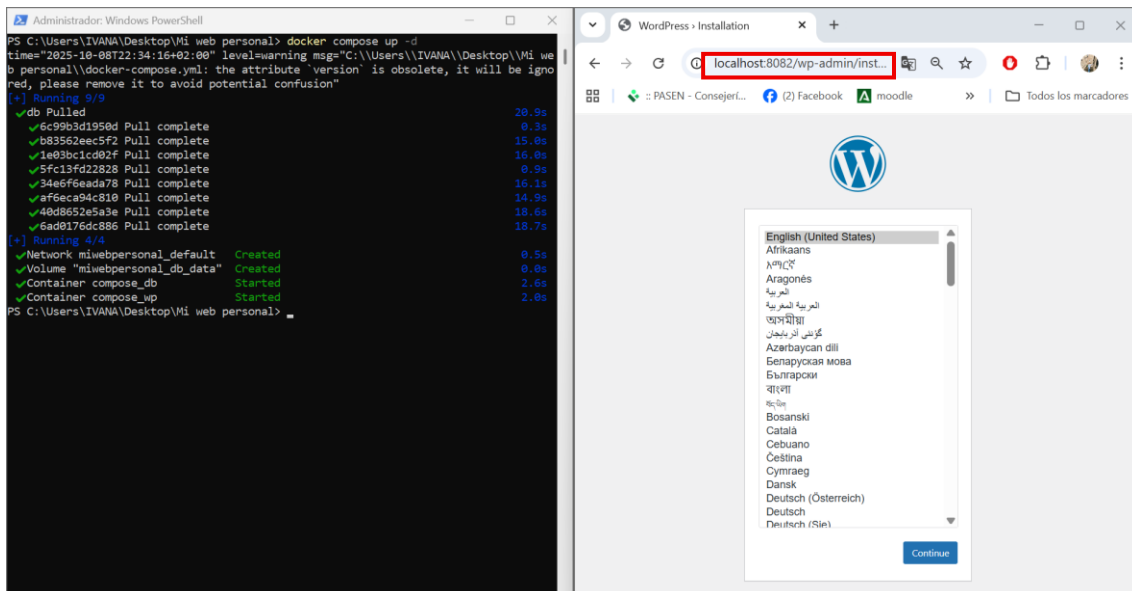
```
version: '3.8'

services:
  db:
    image: mariadb:10.6
    container_name: compose_db
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: test
      MYSQL_DATABASE: bd_wp
      MYSQL_USER: user_wp
      MYSQL_PASSWORD: test
    volumes:
      - db_data:/var/lib/mysql

  wordpress:
    depends_on:
      - db
    image: wordpress:latest
    container_name: compose_wp
    restart: always
    ports:
      - "8082:80"
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: user_wp
      WORDPRESS_DB_PASSWORD: test
      WORDPRESS_DB_NAME: bd_wp

volumes:
  db_data:
```

Para levantar el contenedor utilizaremos el comando: ***docker compose up -d***



The screenshot shows two windows. On the left, a Windows PowerShell window displaying the output of the `docker compose up -d` command. On the right, a web browser window showing the WordPress installation screen.

PowerShell Output:

```
PS C:\Users\IVANA\Desktop\Mi web personal> docker compose up -d
time="2025-10-08T22:34:16+02:00" level=warning msg="C:\\Users\\IVANA\\Desktop\\Mi web personal\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[*] Running 8/9
✓ db Pulled 20.9s
✓ 6e99b3d1956d Pull complete 0.3s
✓ b83562eac5f2 Pull complete 15.8s
✓ 1e03b1cd02f Pull complete 16.8s
✓ 5fc13fd22828 Pull complete 0.9s
✓ 34e6f6eada78 Pull complete 16.1s
✓ afeeca94c810 Pull complete 14.9s
✓ 40d8652e5a3e Pull complete 18.6s
✓ 6ad8176dc886 Pull complete 18.7s
[*] Running 4/4
✓ Network miwebpersonal_default Created 0.5s
✓ Volume "miwebpersonal_db_data" Created 0.8s
✓ Container compose_db Started 2.6s
✓ Container compose_wp Started 2.0s
PS C:\Users\IVANA\Desktop\Mi web personal>
```

WordPress Installation Screen:

The browser window shows the WordPress installation page at `localhost:8082/wp-admin/install.php`. A language selection dropdown is open, showing a list of languages including English (United States), Afrikaans, Azərbaycan dili, and others. The "Continue" button is visible at the bottom of the dropdown.

¿Qué servicios están siendo definidos en este archivo de configuración?

Se definen 2 servicios: db para la base de datos y wordpress para la aplicación web

¿Qué puertos están siendo mapeados para el servicio de Wordpress?

El puerto interno del contenedor (80) se está mapeando al puerto 8082 de mi pc para no interferir con el puerto 80 que ya se está usando.

¿Qué significa el atributo "restart: always" en la definición de ambos servicios?

Que si un contenedor se detiene por cualquier razón, Docker Compose intentará reiniciarlo automáticamente.

¿Qué imagen está siendo utilizada para el servicio de base de datos MySQL?

La imagen de mariadb:10.6

8. Buscar en docker hub la aplicación Portainer y levantar un contenedor. Investigar sobre su funcionamiento, ¿Para que se utiliza esta aplicación?

Portainer es una herramienta de gestión gráfica para Docker. Su principal función es simplificar la administración del entorno Docker. Actúa como un panel de control web que permite a los usuarios gestionar, monitorear y desplegar contenedores, imágenes, volúmenes y redes mediante una interfaz visual, eliminando el uso de los comandos.

docker run -d --name portainer --restart always -p 9443:9443 -v /var/run/docker.sock:/var/run/docker.sock -v portainer_data:/data portainer/portainer-ce:latest

